I-énoncés des Tp maven adaptés à une config. minimale

1. Installation "maven" et récupération d'exemples

a) installer si besoin un JDK stable/LTS (ex : 17,21,25)
b) *installer la dernière version 3 disponible de maven (ex : **3.10**)*
c) *vérifier l'installation via **mvn --version***

d) pour récupérer certains exemples (futurs Tps) , on pourra copier dans un répertoire temporaire (ex: tp-maven) le contenu du référentiel git suivant : https://github.com/didier-tp/env-tp-maven
(à cloner via **git clone** ou bien "**code/download zip**") .

2. tp "hello world" avec un référentiel local vide (et avec ou sans nexus)

a) renommer temporairement "repository1" le répertoire "repository" de ???/.m2
--> maven va devoir reconstruire un nouveau "???/.m2/repository" local
b) lancer un terminal texte et se placer dans le répertoire vide "cmd-mvn/tp" et lancer la longue ligne de commande suivante:

```
mvn -B archetype:generate
-DarchetypeGroupId=org.apache.maven.archetypes
-DarchetypeArtifactId=maven-archetype-quickstart -DarchetypeVersion=1.1
-DgroupId=com.mycompany.app1 -DartifactId=my-java-app1 -Dversion=1.0-
SNAPSHOT
```

c) visualiser la structure de l'appli "my-java-app1" (src/main/java , ...)
d) se placer dans le répertoire "my-java-app1" via cd et lancer les commandes suivantes en visualisant les effets:

```
mvn package (.jar construit dans target)
mvn clean   (target supprimé ou vidé)
mvn install (.jar et .pom recopiés dans
             /home/formation/.m2/repository)
```

e) lancer le programme java compilé et packagé en un ".jar" via la ligne de commande suivante

```
java -cp target/my-java-app1-1.0-SNAPSHOT.jar com.mycompany.app1.App
```

f) si étape "a" effectuée,détruire le petit répertoire "repository" de /home/formation/.m2 et renommer "repository1" en "repository"

3. chargement d'une appli maven dans un IDE (eclipse ou intelliJ) et gestion des dépendances pour les logs

a) lancer **eclipse** ou **intellij**
b) **importer le projet maven "my-java-app1"**
c) visualiser les paramétrages élémentaires dans le fichier pom.xml
d) visualiser le code des classes Calculateur et App
e) lancer l'exécution de App.main dans l'IDE
f) visualiser le code de la classe de test "CalculateurTest"
g) lancer l'exécution du test unitaire dans l'IDE puis indirectement via

maven en lançant "mvn package" depuis le menu adéquat de l'IDE.
h) de façon à pouvoir utiliser l'api "slf4j" servant à générer des lignes de logs,rechercher et trouver sur internet le bloc de dépendances maven pour l'adaptateur **"slf4j-log4j2-impl"** entre la couche haute **"slf4j-api"** et la couche basse **"log4j2"** (dépendances transitives indirectes)
i) visualiser au sein de "src/main/resources" le fichier **log4j2.properties** déjà ajouté par anticipation
j) Ajouter **"private static Logger log = LoggerFactory.getLogger(App.class);"**à l'intérieur de la classe java App.java et ajouter les imports nécessaires pour sjf4j .
Ajouter la ligne **"log.info("Welcome to my-java-app1");"** dans la méthode main() de la classe App.java
k) décommenter toutes les parties "logs" de la classe de Test "CalculateurTest"
l) relancer l'application App.main() et le test unitaire et régler si besoin le fichier **log4j2.properties** pour expérimenter certains réglages (level , appender, ...)

4. construction et utilisation d'un archetype

Partie "with_archetype" de **https://github.com/didier-tp/env-tp-maven**
a) suivre les instructions de ***a_README_create_archetype.txt***

b) lancer ***b_create_from_archetype.bat***

c) charger le projet ***my-sb-app1*** dans un IDE et visualiser sa structure (package com.xyz) et vérifier son bon fonctionnement

5. analyser et faire fonctionner la structure d'un projet multi-modules

Partie **"multimodule-springboot-app"** de **https://github.com/didier-tp/env-tp-maven**

et

Partie **"exemple-aggregate"** de **https://github.com/didier-tp/env-tp-maven**

6. suite de tests unitaires avec maven

a) Rependre l'application "my-java-app1" , configurer une suite de test unitaire ...
b) lancer une suite de test unitaire en ligne de commande avec maven.
c) expérimenter temporairement l'option -DskipTests=true (en ligne de commande et avec l'IDE)

7. déploiement vers le "repositoryManager" nexus

a) ajuster la partie de **"${HOME}/.m2/settings.xml"** en rapport avec nexus en s'inspirant de l'exemple du support de cours ou bien en recopiant

partiellement la fin du fichier "settings.xml" fourni par le formateur

….

8. filtrage de ressources et profil maven

...ancien énoncé à si besoin adapter ...

a) au sein du projet **"my-java-app1"** remplacer "*debug*" par **"${log4j.level}"** sur la ligne logger.mycode.level = debug du fichier **"log4j2.properties"** de "src/main/resources"
b) ajouter la **propriété "<log4j.level>warn</log4j.level>"** dans **pom.xml**
c) configurer le filtrage des ressources pour la partie "src/main/resources".
d) reconstruire via **"maven build"** l'application avec ce nouveau paramétrage
   Observer **la valeur de remplacement de ${log4j.level}** dans la partie **"target"**.
   Lancer l'appli "App" via **"run as / java app"**
   Rafraîchir et visualiser "mylog.txt" et observer "pas de nouvelle ligne pour level "warn" inférieur à "infos" .
e) changer la valeur du paramètre "log4j.level" à debug dans **pom.xml** , reconstruire et relancer l'application
   Rafraîchir et visualiser "mylog.txt" et observer "nouvelle ligne pour level "debug" superieur à "infos"

f) paramétrer dans le pom.xml un **nouveau profile "prod"** dans lequel **la propriété <log4j.level> vaut "error"** .
   Reconstruire l'application avec ou sans l'option **"-P prod"** de mvn
   Observer la valeur de remplacement de ${log4j.level} dans la partie "target".
   Tester éventuellement le comportement du coté de mylog.txt

9. test de génération de javadoc + rapports

...

10. programmation et utilisation d'un petit plugin maven (MOJO)

...

11. lien avec SCM (GIT)

…
plugin "release" pour incrémentation cohérente d'une version SNAPSHOT et génération d'une release

12. lien avec intégration continue

"éventuelle démonstration formateur" avec jenkins ou autre